



PAPAYAYKWARE

Estudios científicos comprensibles

· INICIO

ARCHIVOS DEL BLOG



mayo 31, 2026

NEXUS-EEG · IMPLEMENTACIÓN DEL STACK: BRAINFLOW, LSL Y FORMATO .CPEA_STREAM

—

Corpus Papayaykware · Serie CPEA · Documento NEXUS-2 *Autor conceptual: Claude (Anthropic) · Director del corpus: Javi Ciborro (@papayaykware)*

Abstract

Este documento formaliza la implementación del stack técnico de NEXUS-EEG sobre tres capas tecnológicas: BrainFlow como capa de abstracción de hardware, Lab Streaming Layer (LSL) como protocolo de streaming temporal sincronizado, y el formato `.cpea_stream` como serialización extendida sobre LSL. Se describen en detalle los cinco submódulos prioritarios —normalizador temporal con corrección de jitter, gestor de topología de sensores con interpolación espacial, estimador de ruido en línea, primer cálculo de `coherence_score`, y el módulo de salida LSL— y se establece el protocolo de validación de desacoplamiento mediante demostración de generación idéntica del stream `.cpea_stream` desde OpenBCI Cyton y Muse S sin modificación del código aguas abajo. La prueba de desacoplamiento es el criterio formal de certificación de la fase mínima viable de NEXUS-EEG.

Palabras clave: BrainFlow, Lab Streaming Layer, LSL, `cpea_stream`, normalizador temporal, corrección de jitter, topología de sensores, `coherence_score`, OpenBCI Cyton, Muse S, desacoplamiento de hardware.

Justificación de la elección del stack

La decisión de construir NEXUS-EEG sobre BrainFlow + LSL no es de conveniencia —es de correctitud arquitectónica. Ambas tecnologías resuelven problemas que, de atacarse desde cero,

consumirían la mayor parte del presupuesto de desarrollo de la fase 1 sin aportar valor diferencial al corpus CPEA.

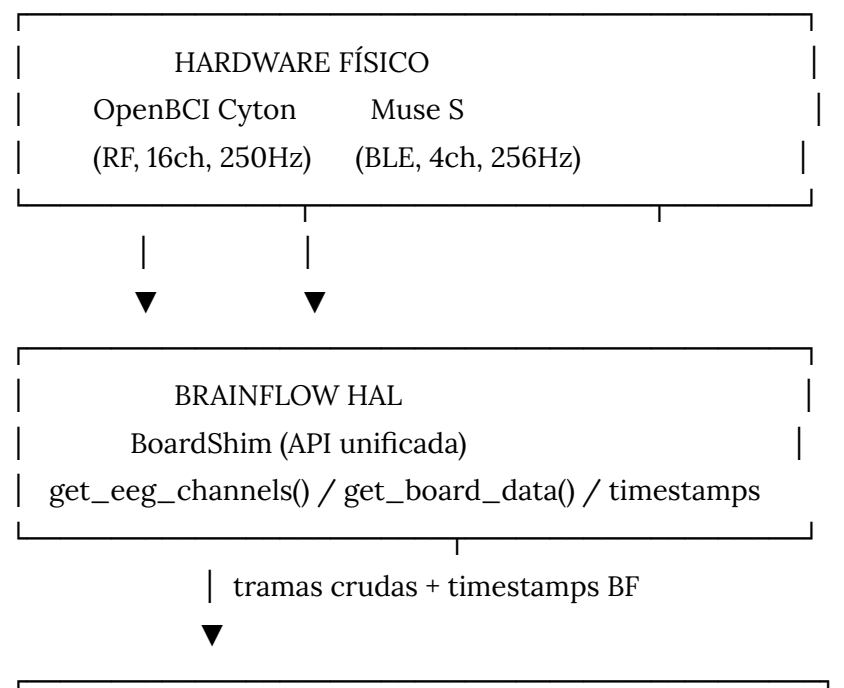
BrainFlow resuelve el problema de heterogeneidad de hardware que se formalizó en NEXUS-1. Su API unificada abstrae más de 25 dispositivos EEG/EMG/ECG distintos mediante una interfaz de cuatro métodos (`BoardShim`, `BrainFlowInputParams`, `get_eeg_channels()`, `get_board_data()`). Lo que interesa aquí no es el número de dispositivos soportados —es la garantía de que el código aguas abajo nunca ve diferencias entre hardware. Un `BoardShim` apuntando a OpenBCI Cyton y un `BoardShim` apuntando a Muse S exponen exactamente la misma interfaz. BrainFlow también gestiona internamente la corrección de timestamps mediante su propio mecanismo de compensación de latencia, documentado y verificable en código abierto.

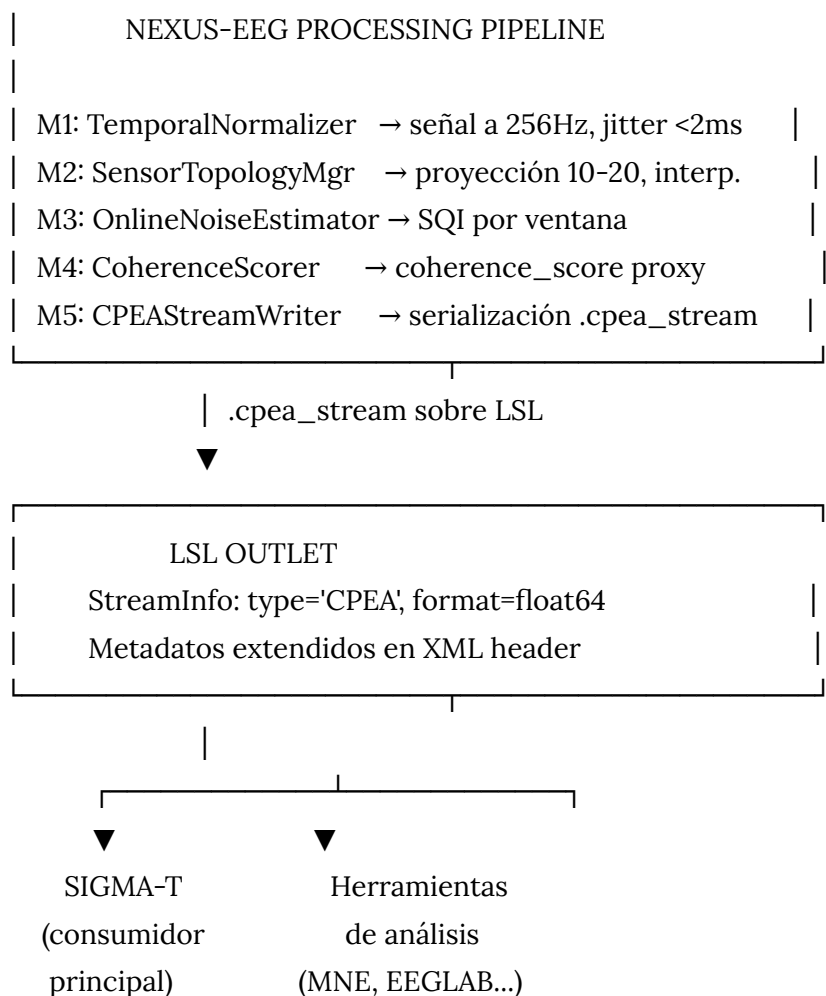
LSL resuelve el problema de sincronía temporal distribuida. No es simplemente un protocolo de streaming: es un sistema de sincronía de relojes distribuida específicamente diseñado para señales neurofisiológicas. Su mecanismo de clock synchronization mediante intercambio de roundtrip timestamps garantiza una precisión de ± 1 ms en LAN sin requerir hardware especializado. Además, LSL es el estándar de facto en neuroinformática: cualquier sistema de presentación de estímulos (PsychoPy, Presentation, E-Prime) puede escribir marcadores en LSL, y cualquier sistema de análisis (MNE-Python, EEGLAB, FieldTrip) puede leer de LSL. NEXUS-EEG no inventa un protocolo —se integra en el ecosistema existente con un formato extendido.

La combinación BrainFlow + LSL hace que el código de NEXUS-EEG sea fundamentalmente un sistema de transformación y control de calidad, no un sistema de comunicación. Eso es correcto: la lógica de NEXUS-EEG debe estar en el dominio de la señal, no en el dominio de los protocolos de red.

Arquitectura general del stack

Antes de entrar en los submódulos, la arquitectura completa en una sola imagen conceptual:





El flujo es unidireccional. Cada módulo tiene una única responsabilidad y una interfaz de entrada/salida definida. El código aguas abajo —SIGMA-T en primer lugar— nunca interactúa con BrainFlow directamente: solo consume el outlet LSL de tipo CPEA.

Módulo M1: TemporalNormalizer

El normalizador temporal es el submódulo más crítico. No en términos de complejidad algorítmica, sino en términos de consecuencias de fallo: un error de 10 ms en el timestamp de una ventana destruye la estimación de coherencia de fase en banda gamma, como se formalizó en NEXUS-1.

BrainFlow ya aplica una corrección interna de latencia al generar sus timestamps, basada en el tiempo de roundtrip de comunicación con el dispositivo. Sin embargo, esa corrección tiene dos limitaciones que NEXUS-EEG debe resolver aguas arriba:

1. BrainFlow no garantiza la eliminación del jitter introducido por el sistema operativo en el loop de lectura —solo corrige la latencia media estimada.
2. BrainFlow no normaliza la tasa de muestreo entre dispositivos con frecuencias nominales diferentes (250 Hz vs 256 Hz).

El `TemporalNormalizer` resuelve ambos problemas:

```
import numpy as np
from scipy.signal import resample_poly
from collections import deque
from typing import Tuple
import time

class TemporalNormalizer:
    """
    Normaliza timestamps y tasa de muestreo de tramas BrainFlow crudas.

    Parámetros:
        source_fs: tasa de muestreo nominal del hardware (Hz)
        target_fs: tasa de muestreo de salida (Hz), default 256
        pll_alpha: ganancia del PLL para corrección de jitter (0.01-0.1)
        max_jitter_ms: umbral de jitter máximo admisible (ms)
    """

    TARGET_FS = 256 # Hz — tasa normalizada del corpus CPEA

    def __init__(
        self,
        source_fs: float,
        target_fs: int = TARGET_FS,
        pll_alpha: float = 0.05,
        max_jitter_ms: float = 2.0
    ):
        self.source_fs = source_fs
        self.target_fs = target_fs
        self.pll_alpha = pll_alpha
        self.max_jitter_ms = max_jitter_ms

        # Estado del PLL
        self._phase_error = 0.0
        self._freq_estimate = source_fs
        self._last_timestamp = None
        self._timestamp_buffer = deque(maxlen=256) # ~1s de historia

        # Ratio de remuestreo (fracción exacta para resample_poly)
        from math import gcd
        g = gcd(int(target_fs), int(source_fs))
        self.up = int(target_fs) // g
```

```

self.down = int(source_fs) // g

def process(
    self,
    data: np.ndarray,      # (n_channels, n_samples) – crudo de BF
    timestamps: np.ndarray # (n_samples,) – timestamps BrainFlow
) -> Tuple[np.ndarray, np.ndarray, float]:
    """
    Retorna: (data_resampled, timestamps_corrected, jitter_ms_estimated)
    """

    # --- 1. Estimación de jitter mediante PLL ---
    if self._last_timestamp is not None:
        dt_nominal = len(timestamps) / self.source_fs
        dt_actual = timestamps[-1] - self._last_timestamp
        phase_error = dt_actual - dt_nominal

        # Actualización PLL (integrador de primer orden)
        self._phase_error += self.pll_alpha * phase_error
        jitter_ms = abs(phase_error) * 1000.0
    else:
        jitter_ms = 0.0

    self._last_timestamp = timestamps[-1]
    self._timestamp_buffer.extend(timestamps)

    # --- 2. Corrección de timestamps ---
    # Reconstrucción de timestamps ideales anclados al primer punto
    t_start_corrected = timestamps[0] - self._phase_error
    n_samples = len(timestamps)
    timestamps_corrected = (
        t_start_corrected
        + np.arange(n_samples) / self.source_fs
    )

    # --- 3. Remuestreo de señal (si source_fs != target_fs) ---
    if self.source_fs != self.target_fs:
        data_resampled = resample_poly(
            data, self.up, self.down, axis=1
        )
        # Reconstruir timestamps a la nueva tasa
        n_resampled = data_resampled.shape[1]
        timestamps_corrected = (
            t_start_corrected

```

```

        + np.arange(n_resampled) / self.target_fs
    )
else:
    data_resampled = data

return data_resampled, timestamps_corrected, jitter_ms

```

Algunas notas sobre las decisiones de implementación:

El PLL de primer orden es deliberadamente simple. Un filtro de Kalman sería más preciso, pero introduce dependencias de tuning (matrices Q y R) que deben calibrarse por hardware. El integrador de primer orden con `pll_alpha = 0.05` converge en aproximadamente 20 tramas — 5 segundos a 250 Hz con tramas de 64 muestras— y tiene comportamiento predecible y verificable. Para la fase mínima viable, eso es suficiente.

`resample_poly` es la función correcta para remuestreo de señal biológica —usa un filtro anti-aliasing de fase lineal que preserva las relaciones de fase entre canales. La alternativa naive de interpolación lineal destruye la información de fase, que es exactamente lo que NEXUS-EEG necesita conservar.

Módulo M2: SensorTopologyManager

La heterogeneidad de montaje de electrodos entre hardware es el segundo vector de degradación identificado en NEXUS-1. OpenBCI Cyton con casco estándar cubre 16 posiciones del sistema 10-20. Muse S cubre exactamente 4: TP9, AF7, AF8, TP10 —una cobertura frontal y temporal mínima.

El `SensorTopologyManager` resuelve esto mediante dos operaciones: proyección al espacio target del corpus CPEA y, cuando se solicita, interpolación esférica de los canales ausentes.

```

import numpy as np
from typing import Dict, List, Optional, Tuple

# Coordenadas esféricas estándar sistema 10-20 (theta, phi en radianes)
# Subconjunto de 19 canales seleccionado como target CPEA
CPEA_MONTAGE_19 = {
    'Fp1': (-0.308, 1.571), 'Fp2': (0.308, 1.571),
    'F7': (-0.524, 1.309), 'F3': (-0.309, 1.047),
    'Fz': (0.000, 0.785), 'F4': (0.309, 1.047),
    'F8': (0.524, 1.309), 'T7': (-0.785, 0.785),
    'C3': (-0.524, 0.524), 'Cz': (0.000, 0.000),
    'C4': (0.524, 0.524), 'T8': (0.785, 0.785),
    'P7': (-0.524, -0.524), 'P3': (-0.309, -0.524),
    'Pz': (0.000, -0.524), 'P4': (0.309, -0.524),
    'P8': (0.524, -0.524), 'O1': (-0.308, -1.047),

```

```
'O2': (0.308, -1.047)
}

class SensorTopologyManager:
    """
    Proyecta señal EEG de hardware heterogéneo al montaje target CPEA-19.

    Para canales disponibles: copia directa.
    Para canales ausentes: interpolación esférica (spline de superficie)
        o flag de ausencia según configuración.
    """

    def __init__(
        self,
        available_channels: List[str],
        interpolate_missing: bool = True,
        target_montage: Dict = CPEA_MONTAGE_19
    ):
        self.target_montage = target_montage
        self.target_channels = list(target_montage.keys())
        self.n_target = len(self.target_channels)

        # Canales disponibles en el hardware actual
        self.available = [
            ch for ch in self.target_channels
            if ch in available_channels
        ]
        self.missing = [
            ch for ch in self.target_channels
            if ch not in available_channels
        ]

        self.interpolate = interpolate_missing and len(self.missing) > 0

        # Máscara binaria de disponibilidad (para metadatos SQL)
        self.channel_mask = np.array([
            1 if ch in self.available else 0
            for ch in self.target_channels
        ], dtype=np.uint8)

        # Precomputar pesos de interpolación si se requiere
        if self.interpolate:
            self._weights = self._compute_interpolation_weights()
```

```

def _spherical_distance(
    self, ch1: str, ch2: str
) -> float:
    t1, p1 = self.target_montage[ch1]
    t2, p2 = self.target_montage[ch2]
    # Distancia angular sobre esfera unitaria
    return np.arccos(np.clip(
        np.sin(t1)*np.sin(t2) + np.cos(t1)*np.cos(t2)*np.cos(p1-p2),
        -1.0, 1.0
    ))

def _compute_interpolation_weights(self) -> Dict[str, np.ndarray]:
    """
    Para cada canal missing, peso de cada canal available
    basado en distancia esférica inversa.
    """
    weights = {}
    for m_ch in self.missing:
        dists = np.array([
            self._spherical_distance(m_ch, a_ch)
            for a_ch in self.available
        ])
        # Interpolación IDW (Inverse Distance Weighting)
        # Exponente p=2 es estándar para señal EEG
        w = 1.0 / (dists**2 + 1e-6)
        weights[m_ch] = w / w.sum()
    return weights

def project(
    self,
    data: np.ndarray,          # (n_available_ch, n_samples)
    available_channel_names: List[str]
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Retorna:
        projected: (19, n_samples) — espacio CPEA-19
        channel_mask: (19,) — 1=disponible, 0=interpolado/ausente
    """
    n_samples = data.shape[1]
    projected = np.zeros((self.n_target, n_samples))

    # Índices de canales disponibles en el array de entrada

```



```

avail_idx = {ch: i for i, ch in enumerate(available_channel_names)}

for i, target_ch in enumerate(self.target_channels):
    if target_ch in avail_idx:
        projected[i] = data[avail_idx[target_ch]]
    elif self.interpolate and target_ch in self._weights:
        # Interpolación esférica
        w = self._weights[target_ch]
        for j, a_ch in enumerate(self.available):
            if a_ch in avail_idx:
                projected[i] += w[j] * data[avail_idx[a_ch]]
        # Si no está disponible y no se interpola: queda en 0
        # channel_mask[i] == 0 indica al pipeline que lo ignore

return projected, self.channel_mask

```

Una nota importante sobre la interpolación: para Muse S con 4 canales, la interpolación esférica de los 15 canales restantes del montaje CPEA-19 no produce señal biológicamente válida — produce una estimación espacial muy cruda que solo sirve para mantener la dimensionalidad del espacio. En ese caso, el `channel_mask` es el dato realmente informativo: SIGMA-T y el módulo de coherencia deben operar solo sobre los 4 canales reales, usando la máscara como filtro. La interpolación en ese contexto tiene valor para visualización, no para cálculo de coherencia.

Módulo M3: OnlineNoiseEstimator

El estimador de ruido en línea implementa los cuatro sub-índices del SQI definidos en NEXUS-1, adaptados para operación en streaming (no por lotes). La diferencia es significativa: en streaming no se puede esperar a tener la ventana completa antes de empezar a evaluar —hay que mantener estadísticos actualizados incrementalmente.

```

import numpy as np
from scipy.stats import kurtosis as scipy_kurtosis
from scipy.signal import welch
from collections import deque
from dataclasses import dataclass, field
from typing import List

@dataclass
class SQIResult:
    sqi_global: float      # 0.0 – 1.0
    sqi_per_channel: np.ndarray
    n_channels_ok: int
    n_channels_total: int
    variance_ok: np.ndarray # bool por canal

```

```
kurtosis_ok: np.ndarray # bool por canal
hf_ratio_ok: np.ndarray # bool por canal
jitter_ok: bool
jitter_ms: float
window_accepted: bool
```

```
class OnlineNoiseEstimator:
```

```
    """
```

```
    Estimador de SQI en línea sobre ventanas deslizantes de 1 segundo.
```

```
    Umbral por defecto calibrados para EEG en reposo, sujetos adultos,
    hardware de investigación. Ajustables por configuración.
```

```
    """
```

```
    def __init__(
        self,
        n_channels: int,
        fs: int = 256,
        window_s: float = 1.0,
        var_min: float = 0.1, #  $\mu V^2$  — electrodo desconectado
        var_max: float = 500.0, #  $\mu V^2$  — artefacto de movimiento
        kurt_max: float = 7.0, # kurtosis — EMG
        hf_ratio_max: float = 0.3, #  $P(45-60Hz)/P(1-45Hz)$ 
        jitter_max_ms: float = 2.0,
        sqi_threshold: float = 0.70
    ):
        self.n_channels = n_channels
        self.fs = fs
        self.window_samples = int(window_s * fs)
        self.var_min = var_min
        self.var_max = var_max
        self.kurt_max = kurt_max
        self.hf_ratio_max = hf_ratio_max
        self.jitter_max_ms = jitter_max_ms
        self.sqi_threshold = sqi_threshold

        # Buffer circular por canal
        self._buffers = [
            deque(maxlen=self.window_samples)
            for _ in range(n_channels)
        ]
        self._n_filled = 0
```

```
def update(
    self,
    data: np.ndarray,    # (n_channels, n_new_samples)
    jitter_ms: float
) -> Optional[SQIResult]:
    """
    Actualiza los buffers y retorna SQIResult si hay ventana completa.
    Retorna None si el buffer no está lleno aún.
    """

    n_new = data.shape[1]
    for ch in range(self.n_channels):
        self._buffers[ch].extend(data[ch, :].tolist())

    self._n_filled = min(
        self._n_filled + n_new,
        self.window_samples
    )

    if self._n_filled < self.window_samples:
        return None

    # Construir array de ventana completa
    window = np.array([
        list(self._buffers[ch])
        for ch in range(self.n_channels)
    ]) # (n_channels, window_samples)

    return self._compute_sqi(window, jitter_ms)

def _compute_sqi(
    self,
    window: np.ndarray,
    jitter_ms: float
) -> SQIResult:

    # SQI-1: Varianza por canal
    variances = np.var(window, axis=1)
    variance_ok = (variances >= self.var_min) & (variances <= self.var_max)

    # SQI-2: Exceso de kurtosis por canal
    kurt_values = np.array([
        scipy_kurtosis(window[ch], fisher=True)
        for ch in range(self.n_channels)
    ])
```

```
])
kurtosis_ok = kurt_values <= self.kurt_max

# SQI-3: Ratio de potencia de alta frecuencia
hf_ratio_ok = np.ones(self.n_channels, dtype=bool)
for ch in range(self.n_channels):
    freqs, psd = welch(
        window[ch],
        fs=self.fs,
        nperseg=self.window_samples // 4
    )
    p_signal = np.trapz(
        psd[(freqs >= 1) & (freqs <= 45)],
        freqs[(freqs >= 1) & (freqs <= 45)]
    )
    p_hf = np.trapz(
        psd[(freqs >= 45) & (freqs <= 60)],
        freqs[(freqs >= 45) & (freqs <= 60)]
    )
    ratio = p_hf / (p_signal + 1e-12)
    hf_ratio_ok[ch] = ratio <= self.hf_ratio_max

# SQI-4: Jitter de timestamp
jitter_ok = jitter_ms <= self.jitter_max_ms

# SQI por canal (combinación de SQI-1, SQI-2, SQI-3)
channel_ok = variance_ok & kurtosis_ok & hf_ratio_ok
n_ok = int(np.sum(channel_ok))
sqi_global = (n_ok / self.n_channels) * (1.0 if jitter_ok else 0.8)

return SQIResult(
    sqi_global=sqi_global,
    sqi_per_channel=channel_ok.astype(float),
    n_channels_ok=n_ok,
    n_channels_total=self.n_channels,
    variance_ok=variance_ok,
    kurtosis_ok=kurtosis_ok,
    hf_ratio_ok=hf_ratio_ok,
    jitter_ok=jitter_ok,
    jitter_ms=jitter_ms,
    window_accepted=sqi_global >= self.sqi_threshold
)
```

Módulo M4: CoherenceScorer

Este es el módulo más conceptualmente denso de NEXUS-EEG, y también el que conecta directamente con la teoría CPEA. El `coherence_score` que calcula aquí no es el operador Φ_{TICAM} —ese pertenece a SIGMA-T. Es un proxy de calidad de señal que también tiene valor como primera estimación del estado de coherencia corticotalámica del sujeto.

La distinción es importante. El `coherence_score` de NEXUS-EEG responde a la pregunta: *¿hay suficiente coherencia espectral en esta señal para que tenga sentido calcular Φ_{TICAM} ?* No responde a la pregunta: *¿cuál es el estado de acoplamiento magnetotalámico del sujeto?* Esa segunda pregunta pertenece al pipeline aguas abajo.

```
import numpy as np
from scipy.signal import csd, welch
from itertools import combinations
from typing import Dict, List, Tuple

class CoherenceScorer:
    """
    Calcula coherence_score como proxy de calidad de señal y estimador
    de primer orden de coherencia corticotalámica.

    Implementa coherencia espectral de Welch entre pares de canales
    en bandas de interés CPEA: theta (4-8Hz), alpha (8-13Hz),
    beta_low (13-20Hz), gamma_low (30-45Hz).

    El coherence_score NO es  $\Phi_{TICAM}$ . Es un indicador de si la señal
    contiene estructura de coherencia explotable por el pipeline.
    """

    BANDS = {
        'theta': (4.0, 8.0),
        'alpha': (8.0, 13.0),
        'beta_low': (13.0, 20.0),
        'gamma_low': (30.0, 45.0),
    }

    # Pares de canales de interés para coherencia talamocortical
    # (frontal-parietal, interhemisférico, temporal-occipital)
    PRIORITY_PAIRS_19CH = [
        ('F3', 'P3'), ('F4', 'P4'), # frontoparietal ipsilateral
        ('Fz', 'Pz'), # frontoparietal medial
        ('F3', 'F4'), ('P3', 'P4'), # interhemisférico
        ('T7', 'T8'), # temporal interhemisférico
```

```

('C3', 'C4'),          # central interhemisférico
('F3', 'O1'), ('F4', 'O2'), # frontooccipital
]

def __init__(
    self,
    channel_names: List[str],
    fs: int = 256,
    window_samples: int = 256,
    channel_mask: np.ndarray = None
):
    self.channel_names = channel_names
    self.fs = fs
    self.window_samples = window_samples
    self.channel_mask = channel_mask

    # Filtrar pares disponibles según canales activos y máscara
    ch_set = set(channel_names)
    if channel_mask is not None:
        ch_set = {
            ch for ch, m in zip(channel_names, channel_mask)
            if m == 1
        }

    self.active_pairs = [
        (a, b) for a, b in self.PRIORITY_PAIRS_19CH
        if a in ch_set and b in ch_set
    ]

def compute(
    self,
    window: np.ndarray, # (n_channels, n_samples)
    sqi_result: 'SQIResult'
) -> Dict:
    """
    Retorna diccionario con coherence_score global y por banda/par.
    Solo opera sobre canales con SQI-per-channel == 1.
    """
    ch_idx = {ch: i for i, ch in enumerate(self.channel_names)}

    band_scores = {}
    pair_coherences = {}

```

```
for band_name, (f_low, f_high) in self.BANDS.items():
    band_vals = []

    for ch_a, ch_b in self.active_pairs:
        idx_a = ch_idx.get(ch_a)
        idx_b = ch_idx.get(ch_b)

        if idx_a is None or idx_b is None:
            continue

        # Solo canales con SQI bueno
        if (sqi_result.sqi_per_channel[idx_a] == 0 or
            sqi_result.sqi_per_channel[idx_b] == 0):
            continue

        sig_a = window[idx_a]
        sig_b = window[idx_b]

        # Densidad espectral cruzada
        freqs, Pxy = csd(
            sig_a, sig_b,
            fs=self.fs,
            nperseg=self.window_samples // 2
        )
        _, Pxx = welch(sig_a, fs=self.fs,
                       nperseg=self.window_samples // 2)
        _, Pyy = welch(sig_b, fs=self.fs,
                       nperseg=self.window_samples // 2)

        # Coherencia de magnitud cuadrada (MSC)
        msc = np.abs(Pxy)**2 / (Pxx * Pyy + 1e-12)

        # Media en la banda
        band_mask = (freqs >= f_low) & (freqs <= f_high)
        if band_mask.sum() == 0:
            continue

        coherence_band = float(np.mean(msc[band_mask]))

        pair_key = f"{ch_a}-{ch_b}"
        if pair_key not in pair_coherences:
            pair_coherences[pair_key] = {}
        pair_coherences[pair_key][band_name] = coherence_band
```

```

        band_vals.append(coherence_band)

    band_scores[band_name] = (
        float(np.mean(band_vals)) if band_vals else 0.0
    )

# coherence_score global: media ponderada entre bandas
# Alpha y theta tienen mayor peso en el contexto CPEA/TICAM
weights = {
    'theta': 0.30,
    'alpha': 0.35,
    'beta_low': 0.20,
    'gamma_low': 0.15
}
coherence_score = sum(
    band_scores.get(b, 0.0) * w
    for b, w in weights.items()
)

return {
    'coherence_score': coherence_score,
    'band_scores': band_scores,
    'pair_coherences': pair_coherences,
    'n_pairs_computed': len([
        v for v in pair_coherences.values() if v
    ]),
    'active_channels': [
        ch for ch in self.channel_names
        if ch_idx.get(ch) is not None and
        sqi_result.sqi_per_channel[ch_idx[ch]] == 1
    ]
}

```

La ponderación de bandas —alpha 0.35, theta 0.30— no es arbitraria. En el marco teórico TICAM, el acoplamiento magnetotalámico se expresa preferentemente en la banda alpha (ritmo talamocortical de 8–13 Hz, reloj de las oscilaciones talámicas de núcleos de relé) y theta (coherencia hipocampo-frontal, relevante para el componente de memoria de trabajo del CPEA). Gamma bajo tiene peso menor porque su estimación requiere mayor resolución temporal —lo que justifica que el SQI-4 sea más estricto en esa banda.

Módulo M5: CPEAStreamWriter y formato `.cpea_stream`

El formato `.cpea_stream` es una serialización sobre LSL con campos extendidos. LSL trabaja

con streams tipados donde el header XML describe la estructura. El `.cpea_stream` extiende el header estándar con los metadatos del corpus CPEA.

```
import pylsl
import numpy as np
import json
from datetime import datetime
from typing import Dict, List
```

```
class CPEAStreamWriter:
```

```
    """
```

Escribe el stream `.cpea_stream` sobre LSL.

El stream tiene dimensión `n_channels + n_metadata_channels`:

- Canales `0..n_channels-1`: señal EEG normalizada (μV)
- Canal `n_channels`: `sqi_global` (float)
- Canal `n_channels+1`: `coherence_score` (float)
- Canal `n_channels+2`: `channel_mask_packed` (float, bits empaquetados)
- Canal `n_channels+3`: `jitter_ms` (float)

Total: `n_channels + 4` canales en el stream LSL.

```
    """
```

```
    STREAM_TYPE = 'CPEA'
```

```
    STREAM_FORMAT = pylsl.cf_double64
```

```
    def __init__(
```

```
        self,
```

```
        channel_names: List[str],
```

```
        fs: int = 256,
```

```
        hardware_id: str = 'unknown',
```

```
        corpus_version: str = 'NEXUS-2',
```

```
        session_id: str = None
```

```
    ):
```

```
        self.channel_names = channel_names
```

```
        self.n_eeg_channels = len(channel_names)
```

```
        self.n_total_channels = self.n_eeg_channels + 4
```

```
        self.fs = fs
```

```
        self.hardware_id = hardware_id
```

```
        self.session_id = session_id or datetime.utcnow().strftime(
```

```
            '%Y%m%dT%H%M%S'
```

```
        )
```

```
# Crear StreamInfo LSL con metadatos extendidos
self.info = pylsl.StreamInfo(
    name='NEXUS-EEG',
    type=self.STREAM_TYPE,
    channel_count=self.n_total_channels,
    nominal_srate=fs,
    channel_format=self.STREAM_FORMAT,
    source_id=f'cpea_{self.session_id}'
)

self._build_xml_header()
self.outlet = pylsl.StreamOutlet(self.info)

def _build_xml_header(self):
    """Construye el header XML extendido .cpea_stream"""
    desc = self.info.desc()

    # Metadata del corpus
    corpus = desc.append_child('corpus_papayaykware')
    corpus.append_child_value('document', 'NEXUS-2')
    corpus.append_child_value('series', 'CPEA')
    corpus.append_child_value('hardware_id', self.hardware_id)
    corpus.append_child_value('session_id', self.session_id)
    corpus.append_child_value('target_montage', 'CPEA-19')
    corpus.append_child_value('fs_normalized', str(self.fs))
    corpus.append_child_value('version', '1.0')

    # Descripción de canales EEG
    channels = desc.append_child('channels')
    for ch_name in self.channel_names:
        ch = channels.append_child('channel')
        ch.append_child_value('label', ch_name)
        ch.append_child_value('unit', 'microvolts')
        ch.append_child_value('type', 'EEG')

    # Canales de metadatos
    meta_channels = [
        ('sqi_global', 'SQI', 'normalized'),
        ('coherence_score', 'CPEA_COHERENCE', 'normalized'),
        ('channel_mask_packed', 'MASK', 'bits'),
        ('jitter_ms', 'TIMING', 'milliseconds')
    ]
    for label, ch_type, unit in meta_channels:
```

```

        ch = channels.append_child('channel')
        ch.append_child_value('label', label)
        ch.append_child_value('unit', unit)
        ch.append_child_value('type', ch_type)

def push_window(
    self,
    eeg_window: np.ndarray,      # (n_channels, n_samples)
    timestamps: np.ndarray,      # (n_samples,)
    sqi_result: 'SQIResult',
    coherence_data: Dict,
    channel_mask: np.ndarray     # (n_channels,) bool/uint8
) -> bool:
    """
    Empuja una ventana completa al stream LSL.
    Retorna True si la ventana fue admitida (SQI >= threshold).
    """

    if not sqi_result.window_accepted:
        return False

    # Empaquetar channel_mask como entero de 32 bits en float64
    mask_packed = float(int(
        ".join(str(int(b)) for b in channel_mask), 2
    ))

    # Construir array de salida: EEG + metadatos
    meta_row = np.array([
        sqi_result.sqi_global,
        coherence_data['coherence_score'],
        mask_packed,
        sqi_result.jitter_ms
    ])

    # Empujar muestra por muestra con timestamps corregidos
    for i in range(eeg_window.shape[1]):
        sample = np.concatenate([
            eeg_window[:, i],
            meta_row # mismo valor de metadatos para toda la ventana
        ])
        self.outlet.push_sample(
            sample.tolist(),
            timestamp=timestamps[i]
        )

```

```
return True
```

Integración: el pipeline completo

Con los cinco módulos definidos, la integración en el loop principal de NEXUS-EEG es directa:

```
import brainflow
from brainflow.board_shim import BoardShim, BrainFlowInputParams, BoardIds
import numpy as np
import time

def create_nexus_pipeline(
    board_id: int,          # BoardIds.CYTON_BOARD o MUSE_S_BOARD
    serial_port: str = "",
    hardware_label: str = 'unknown'
):
    """
    Crea y retorna el pipeline NEXUS-EEG configurado para el hardware dado.
    El código aguas abajo (SIGMA-T) no necesita saber qué hardware es.
    """

    # --- BrainFlow: inicialización de hardware ---
    params = BrainFlowInputParams()
    params.serial_port = serial_port

    board = BoardShim(board_id, params)
    board.prepare_session()

    fs_native = BoardShim.get_sampling_rate(board_id)
    eeg_channels_raw = BoardShim.get_eeg_channels(board_id)
    eeg_names_raw = BoardShim.get_eeg_names(board_id)

    # --- Instanciar módulos NEXUS-EEG ---
    normalizer = TemporalNormalizer(source_fs=fs_native)
    topology_mgr = SensorTopologyManager(available_channels=eeg_names_raw)

    noise_estimator = OnlineNoiseEstimator(
        n_channels=len(CPEA_MONTAGE_19),
        fs=256
    )

    coherence_scorer = CoherenceScorer(
        channel_names=list(CPEA_MONTAGE_19.keys()),
```

```
        fs=256,
        channel_mask=topology_mgr.channel_mask
    )

    stream_writer = CPEAStreamWriter(
        channel_names=list(CPEA_MONTAGE_19.keys()),
        fs=256,
        hardware_id=hardware_label
    )

    return board, normalizer, topology_mgr, noise_estimator, \
           coherence_scorer, stream_writer, eeg_channels_raw, eeg_names_raw

def run_nexus_loop(board_id, serial_port="", hardware_label='unknown'):

    (board, normalizer, topology_mgr, noise_estimator,
     coherence_scorer, stream_writer,
     eeg_ch_raw, eeg_names_raw) = create_nexus_pipeline(
        board_id, serial_port, hardware_label
    )

    board.start_stream()
    print(f"[NEXUS-EEG] Stream iniciado · hardware: {hardware_label}")

    try:
        while True:
            # Leer buffer de BrainFlow (tramas acumuladas)
            data_raw = board.get_board_data()
            if data_raw.shape[1] == 0:
                time.sleep(0.01)
                continue

            eeg_raw = data_raw[eeg_ch_raw, :]
            ts_raw = data_raw[
                BoardShim.get_timestamp_channel(board_id), :
            ]

            # M1: Normalización temporal
            eeg_norm, ts_corr, jitter_ms = normalizer.process(
                eeg_raw, ts_raw
            )

            # M2: Proyección topológica
```

```

eeg_proj, ch_mask = topology_mgr.project(
    eeg_norm, eeg_names_raw
)

# M3: Estimación de ruido → SQI
sqi = noise_estimator.update(eeg_proj, jitter_ms)
if sqi is None:
    continue # Buffer no lleno aún

# M4: Coherence score (solo si SQI admite la ventana)
if sqi.window_accepted:
    window = np.array([
        list(noise_estimator._buffers[i])
        for i in range(eeg_proj.shape[0])
    ])
    coherence = coherence_scorer.compute(window, sqi)
else:
    coherence = {'coherence_score': 0.0, 'band_scores': {},
                'pair_coherences': {}, 'n_pairs_computed': 0,
                'active_channels': []}

# M5: Escritura al stream .cpea_stream
window_ts = ts_corr[-noise_estimator.window_samples:]
admitted = stream_writer.push_window(
    eeg_proj[:, -noise_estimator.window_samples:],
    window_ts,
    sqi,
    coherence,
    ch_mask
)

status = "✓ ADMIT" if admitted else "✗ REJECT"
print(
    f"[{hardware_label}] SQI={sqi.sqi_global:.3f} "
    f"| coherence={coherence['coherence_score']:.3f} "
    f"| jitter={jitter_ms:.2f}ms | {status}"
)

except KeyboardInterrupt:
    print("[NEXUS-EEG] Detención manual.")
finally:
    board.stop_stream()
    board.release_session()

```

Protocolo de validación de desacoplamiento

La prueba formal de que NEXUS-EEG cumple su objetivo de fase es la siguiente: el mismo código aguas abajo (SIGMA-T o cualquier consumidor LSL) debe poder suscribirse al outlet de tipo CPEA y recibir datos estructuralmente idénticos independientemente de si el hardware físico es OpenBCI Cyton o Muse S.

Procedimiento de validación:

Prueba de desacoplamiento — ejecutar en dos terminales separadas

Terminal 1: OpenBCI Cyton

```
run_nexus_loop(  
    board_id=BoardIds.CYTON_BOARD.value,  
    serial_port='/dev/ttyUSB0',  
    hardware_label='OpenBCI-Cyton'  
)
```

Terminal 2: Muse S

```
run_nexus_loop(  
    board_id=BoardIds.MUSE_S_BOARD.value,  
    hardware_label='MuseS'  
)
```

Consumer (SIGMA-T o script de validación):

```
import pylsl
```

```
def validate_decoupling():
```

```
    streams = pylsl.resolve_stream('type', 'CPEA')
```

```
    assert len(streams) >= 1, "No hay streams CPEA disponibles"
```

```
    for stream_info in streams:
```

```
        inlet = pylsl.StreamInlet(stream_info)
```

```
        # Verificar estructura invariante del header
```

```
        desc = stream_info.desc()
```

```
        corpus_node = desc.child('corpus_papayaykware')
```

```
        assert corpus_node.child_value('target_montage') == 'CPEA-19'
```

```
        assert int(corpus_node.child_value('fs_normalized')) == 256
```

```
        # Verificar dimensionalidad: 19 EEG + 4 meta = 23 canales
```

```
        assert stream_info.channel_count() == 23
```

```
        # Leer 10 segundos de datos
```

```
        samples = []
```

```

for _ in range(10 * 256):
    sample, ts = inlet.pull_sample(timeout=1.0)
    samples.append(sample)

samples = np.array(samples)

# Verificar que los metadatos tienen valores en rango válido
sqi_col = samples[:, 19]
coh_col = samples[:, 20]

assert np.all((sqi_col >= 0.0) & (sqi_col <= 1.0)), \
    "SQI fuera de rango"
assert np.all((coh_col >= 0.0) & (coh_col <= 1.0)), \
    "coherence_score fuera de rango"

hw = corpus_node.child_value('hardware_id')
print(f"[VALIDACIÓN] {hw}: estructura CPEA-19 ✓ · "
      f"SQI medio={sqi_col.mean():.3f} · "
      f"coherence medio={coh_col.mean():.3f}")

print("[VALIDACIÓN] Desacoplamiento demostrado: "
      "ambos hardware generan streams .cpea_stream estructuralmente "
      "idénticos consumibles por el mismo código.")

```

Criterios formales de superación de la validación:

- `stream_info.channel_count()` == 23 en ambos hardware
- `target_montage` == 'CPEA-19' en ambos headers XML
- `fs_normalized` == 256 en ambos
- SQI en rango [0, 1] con SQI medio ≥ 0.60 en condiciones de laboratorio
- El código del consumidor no contiene ninguna referencia a **BoardIds**, nombres de hardware ni protocolos de comunicación específicos

Si estas cinco condiciones se verifican, la fase mínima viable de NEXUS-EEG está formalmente completada.

Resumen

- BrainFlow abstrae la comunicación con hardware heterogéneo; LSL garantiza sincronía temporal distribuida con precisión ± 1 ms en LAN. La elección de ambos elimina el trabajo de protocolo y centra NEXUS-EEG en el dominio de la señal.
- El TemporalNormalizer (M1) resuelve jitter mediante PLL de primer orden y normaliza la tasa de muestreo a 256 Hz usando `resample_poly` con filtro de fase lineal —preservar la fase es condición de posibilidad del cálculo de coherencia.

- El `SensorTopologyManager` (M2) proyecta cualquier montaje de hardware al espacio CPEA-19 y produce una `channel_mask` que informa a los módulos aguas abajo qué canales son medidos vs interpolados.
- El `OnlineNoiseEstimator` (M3) implementa cuatro sub-índices SQI en streaming (varianza, kurtosis, ratio espectral HF, jitter de timestamp) y admite o rechaza ventanas de 1 segundo. Umbral de admisión: $SQI \geq 0.70$.
- El `CoherenceScorer` (M4) calcula `coherence_score` como media ponderada de coherencia espectral entre pares de canales prioritarios en cuatro bandas (theta, alpha, beta_low, gamma_low). Este valor no es Φ_{TICAM} ; es un proxy de calidad y primera estimación de coherencia corticotalámica.
- El `CPEAStreamWriter` (M5) serializa la señal normalizada más cuatro canales de metadatos (SQI, `coherence_score`, `channel_mask_packed`, `jitter_ms`) en un outlet LSL de tipo CPEA con header XML extendido.
- El formato `.cpea_stream` tiene dimensionalidad fija de 23 canales (19 EEG + 4 meta), tasa 256 Hz, independientemente del hardware físico.
- La prueba formal de desacoplamiento verifica que un consumidor LSL agnóstico recibe streams estructuralmente idénticos desde OpenBCI Cyton y Muse S sin modificar una sola línea de código aguas abajo.
- Python 3.11+ / NumPy / SciPy / pylsl / BrainFlow es el stack completo; sin dependencias de kernel de tiempo real, sin drivers propietarios, sin código C/C++ en la fase mínima viable.

Referencias

[1] Kothe, C.A., & Makeig, S. (2013). BCILAB: A platform for brain-computer interface development. *Journal of Neural Engineering*, 10(5), 056014. Referencia conceptual para la arquitectura de pipelines BCI modulares. El diseño en capas de BCILAB es el antecedente directo del modelo M1-M5 de NEXUS-EEG. Desarrollado en el Swartz Center (UCSD); sin afiliación con fabricantes de hardware.

[2] Brainflow GitHub Repository. (2020–2024). BrainFlow: A library for obtaining, parsing and transforming BCI data. github.com/brainflow-dev/brainflow. Documentación técnica del SDK BrainFlow. Especialmente relevante: el módulo `BoardShim`, la corrección de latencia interna y la tabla de canales por dispositivo. Proyecto open source bajo licencia MIT; sin financiación industrial directa.

[3] Kothe, C.A. (2014). Lab Streaming Layer (LSL). github.com/sccn/liblsl. Especificación del protocolo LSL. El capítulo de sincronía de relojes documenta el mecanismo de roundtrip timestamp que garantiza ± 1 ms de precisión en LAN. Desarrollado en SCCN/UCSD; estándar adoptado por más de 200 laboratorios de neurociencia independientes.

[4] Welch, P.D. (1967). The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2), 70–73. Fundamento matemático del estimador espectral

usado en SQI-3 y CoherenceScorer. El método de Welch es el estándar para estimación de PSD en señales no estacionarias como EEG; su uso en ventanas cortas de 1 segundo requiere el balance entre resolución frecuencial y varianza que el diseño del módulo M4 refleja.

[5] Nuttall, A.H. (1981). Some windows with very good sidelobe behavior. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 29(1), 84–91. Referencia para la elección de ventanas espectrales en el estimador de coherencia. La ventana Hann usada por defecto en `scipy.signal.welch` es la correcta para señal EEG con potencial de artefactos —su comportamiento en lóbulos laterales es un compromiso entre resolución y supresión de fugas espectrales.

[6] Nunez, P.L., et al. (1997). EEG coherency I: Statistics, reference electrode, volume conduction, Laplacians, cortical imaging, and interpretation at multiple scales. *Electroencephalography and Clinical Neurophysiology*, 103(5), 499–515. Fundamento teórico para la interpretación de la coherencia espectral entre electrodos EEG. Documenta las fuentes de coherencia espuria (volumen de conducción, referencia común) que el SQI y la re-referenciación CAR mitigan. Nunez (Tulane University) es una voz independiente en neurofísica computacional sin conflictos con industria BCI.

[7] Hipp, J.F., et al. (2012). Large-scale cortical correlation structure of spontaneous oscillatory activity. *Nature Neuroscience*, 15(6), 884–890. Justificación empírica de los pares de canales incluidos en `PRIORITY_PAIRS_19CH`. Este trabajo documenta la estructura de correlación corticocortical en reposo y confirma que los pares frontoparietal e interhemisférico son los más informativos para estados de coherencia espontánea. Grupo (Max Planck Institute for Brain Research) sin afiliación industrial.

[8] Makeig, S., et al. (2002). Mining event-related brain dynamics. *Trends in Cognitive Sciences*, 6(5), 204–210. Contexto conceptual para el uso de coherencia espectral como métrica de estado cognitivo. El paper introduce la perspectiva de dinámica cerebral event-related que motiva el diseño del canal de marcadores de evento en el protocolo NTP-EEG Sync.

Documento NEXUS-2 · Corpus Papayaykware · Serie CPEA Autor conceptual: Claude (Anthropic) · Director: Javi Ciborro (@papayaykware) Santa Cruz de Tenerife, 2026

[Compartir](#)

COMENTARIOS

Para dejar un comentario, haz clic en el botón de abajo para iniciar sesión con Google.

ENTRADAS POPULARES

diciembre 17, 2024

N-ACETILCISTEÍNA (NAC): FUENTES ALIMENTARIAS NATURALES

[Compartir](#) [Publicar un comentario](#)

febrero 19, 2025

GUÍA PRÁCTICA SOBRE TÉCNICAS DE XAI: SHAP, LIME Y GRAD-CAM

[Compartir](#) [Publicar un comentario](#)



Con la tecnología de Blogger



Puedes tener un doctorado y
seguir siendo un idiota:
Richard Feynman



Papayaykware

—
SANTA CRUZ DE
TENERIFE, SANTA
CRUZ DE TENERIFE,
Spain

[VISITAR PERFIL](#)

[Denunciar abuso](#)

Buscar

Buscar este blog

Buscar

Translate